후루하타 라나와 배우는

컴퓨터과학

라나
프로그램이 어떻게 움직일지 생각하고 있었습니다.
선생님
라나, 뭐 하고 있어?

라나
……같이 해보시겠어요?
선생님
나도 보면 알 수 있으려나.

라나
네. 종이에 적으면서 따라가 볼 수도 있으니까요.
선생님
컴퓨터도 없이?

라나
우선은 간단한 것부터 해보시겠어요?
선생님
좋아. 어디서부터 보면 돼?

유한 오토마타

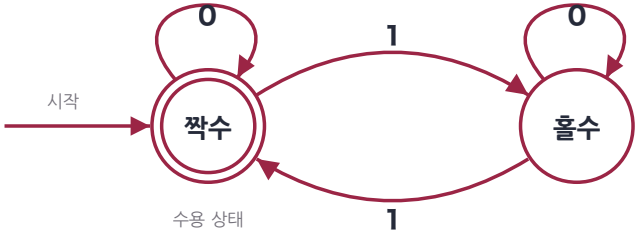
라나 선생님. 0과 1로 된 문자열에 1이 짝수 개 들어 있는지, 어떻게 확인하면 될까요?

선생님 하나씩 세면 되겠네?

라나 네. 그런데…… 짝수인지 홀수인지만 알면 되니, 정확히 몇 개인지는 몰라도 됩니다.

라나 1을 하나 읽을 때마다 짝수, 홀수, 짝수…… 이렇게 바뀌니까요.

01 1을 읽으면, 반대쪽으로



화살표의 숫자는 읽을 문자, 이중 원은 수용 상태입니다.

라나 이걸 각각 하나의 상태로 나타내면 이렇게 됩니다. 1을 읽을 때마다 반대쪽으로 옮겨가는 겁니다.

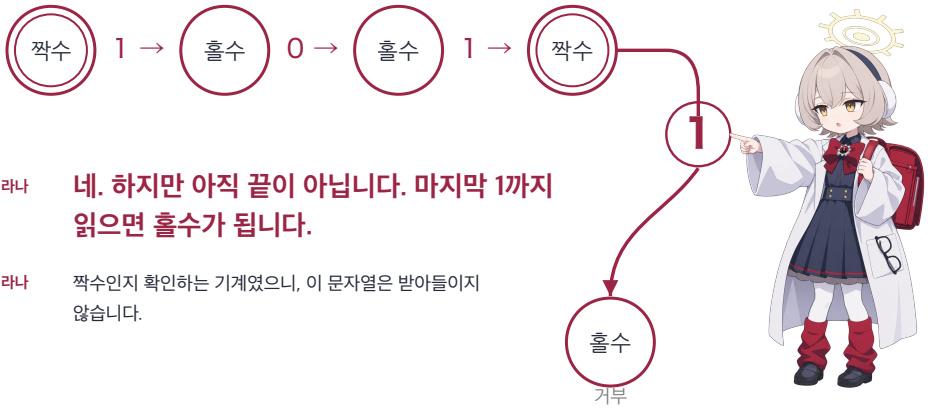
선생님 0을 읽으면?

라나 그대로입니다. 1의 개수는 늘어나지 않았으니까요.

라나 처음에는 여기, 짝수에서 시작합니다. 아직 읽은 1이 없으니 0개입니다.

라나 1011을 넣어보겠습니다. 처음은 짝수. 1을 읽으면 홀수, 다음은 0이니 그대로…….

선생님 그다음이 10이니까, 다시 짝수네.



라나 **네. 하지만 아직 끝이 아닙니다. 마지막 1까지 읽으면 홀수가 됩니다.**

라나 짝수인지 확인하는 기계였으니, 이 문자열은 받아들이지 않습니다.

선생님 중간에 짝수가 됐다고 멈추면 안 되겠구나.

라나 네. 뒤에 10이 더 있을지도 모르니까요.

라나 **이처럼 정해진 수의 상태 사이를 옮겨 다니며 입력을 읽는 기계를 유한 오토마타라고 합니다.**

선생님 문자열이 훨씬 길어져도, 상태는 두 개면 돼?

라나 네. 읽는 데는 더 오래 걸리겠지만, 상태를 더 만들 필요는 없습니다.

$$0*(10*10*)*$$

라나 이 기계가 받아들이는 문자열들을 정규표현식으로 적으면 이렇게 됩니다.

라나 괄호 안에 10이 두 개 있지요. 이 묶음을 반복하면 1은 늘 짝수 개가 됩니다. 사이에 0이 몇 개 들어가든 상관없고요.

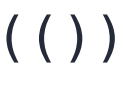
선생님 아무것도 입력하지 않으면?

라나 그것도 받아들입니다. 처음의 짝수 상태에서 움직이지 않았으니까요.

라나 ……입력하는 걸 잊으신 건지, 일부러 비워두신 건지는 이 기계로 알 수 없습니다만.

※ 여기서 정규표현식은 형식언어 이론의 의미입니다. 역참조 등의 확장은 포함하지 않습니다.

괄호를 닫는 순서



수용

거부

라나 선생님. 이번에는 괄호입니다. 이 두 문자열은 여는 괄호와 닫는 괄호의 개수가 같은데, 둘 다 괜찮을까요?

선생님 두 번째는 괄호의 순서가 맞지 않네.

라나 네. 개수만 맞춰서는 안 됩니다. 괄호가 열리고 닫히는 순서가 올바른지도 확인해야 합니다.

아직 닫지 않은 괄호의 수

선생님 그 개수를 상태로 만들면 되지 않아? 0개, 1개, 2개…… 이렇게.

라나 네. 괄호가 두 겹까지만 들어온다면 그렇게 만들 수 있습니다.

선생님 세 겹이면 상태를 하나 더 만들고?

라나 네. 그런데 몇 겹까지 들어올지 정하지 않으면, 상태를 어디까지 만들어 두어야 할까요?

선생님 아무리 많이 만들어도 모자랄 수 있겠네.

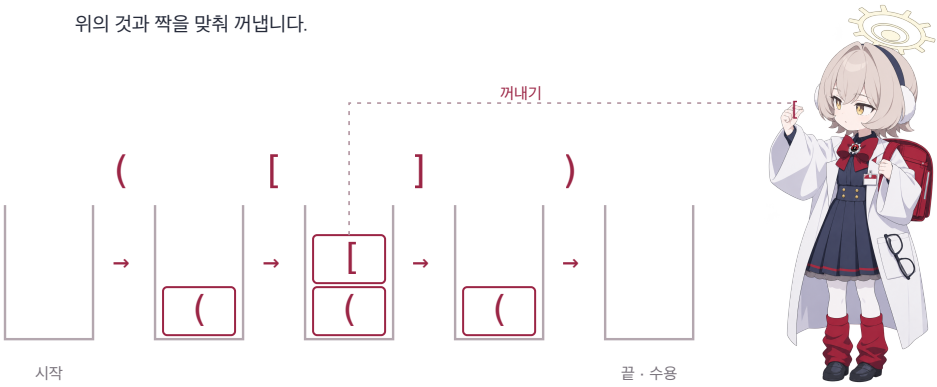
라나 **맞습니다. 깊이에 제한이 없는 괄호의 짝은, 유한 오토마타만으로는 전부 확인할 수 없습니다.**

라나 이번에는 아직 닫지 않은 괄호를 따로 쌓아 두겠습니다.

선생님 상태 말고도 기억할 곳을 하나 더 두는 거구나.

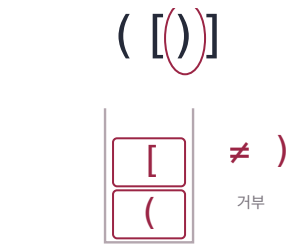
라나 네. 이제 괄호가 늘어나면, 쌓아 두는 쪽이 높아집니다.

라나 이번에는 대괄호도 넣어 보겠습니다. 여는 괄호는 위에 올리고, 닫는 괄호는 맨 위의 것과 짝을 맞춰 꺼냅니다.



선생님 나중에 연 대괄호부터 닫히네.

라나 **네. 이렇게 맨 위에 넣고 맨 위에서 꺼내는 저장 구조를 스택이라고 합니다.**



선생님 그럼 이쪽은? 괄호 종류별로 개수는 맞는데.

라나 지금 닫으려는 것은 소괄호인데, 맨 위에는 대괄호가 남아 있습니다. 그걸 먼저 닫아야 합니다.

라나 꺼낼 괄호가 없거나 종류가 다르면 거부합니다. 끝까지 읽고도 남아 있으면, 그것도 짝이 맞지 않은 겁니다.

) 꺼낼 괄호 없음

(( ) 끝에 (가 남음

선생님 중간에 틀린 곳이 없고, 끝에서 비어 있으면 되는 거네.

라나 네. 유한 오토마타에 이런 스택을 붙인 기계를 푸시다운 오토마타라고 합니다.

선생님 괄호 하나 빠져도 안 넘어가 주는구나.

라나 네. 선생님께서 어디까지 묶으려 하셨는지는, 제가 마음대로 정할 수 없으니까요.

※ 이론에서는 스택의 크기에 제한이 없습니다. 소괄호 한 종류만 확인한다면, 음수가 되는지 검사하는 카운터로도 충분합니다.

글자를 읽어서 계산하기

2 + 3 * 4

*는 곱하기

라나 선생님. 이번에는 이 식을 컴퓨터에게 계산시켜 보겠습니다. 무엇부터 해야 할까요?

선생님 곱셈부터 하면 되겠네. 답은 14고.

라나 네. 그런데 컴퓨터에는 우선 글자가 들어옵니다. 어느 것이 숫자이고, 어느 것이 계산 기호인지부터 구분해 줘야 합니다.

숫자 2	+	숫자 3	*	숫자 4

12
↓
숫자 12

라나 이렇게 읽을 단위로 나눈 것을 토큰이라고 합니다. 숫자는 숫자끼리, 기호는 기호대로 알아보는 과정이 어휘 분석입니다.

선생님 그냥 한 글자씩 나누는 건 아니구나.

라나 네. 12라면 두 글자를 모아서 숫자 하나로 읽습니다. 여기서는 띄어쓰기도 건너뛰고요.

2 + 3 * 4

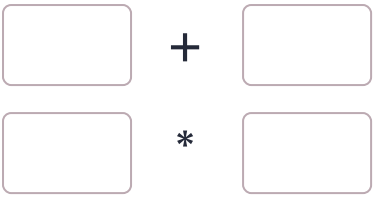
먼저 묶기

선생님 이제 왼쪽부터 계산하면 돼?

라나 그러면 2와 3을 먼저 더해서, 답이 20이 됩니다. 아까 선생님께서는 곱셈부터 하셨지요.

선생님 컴퓨터한테도 그 순서를 알려줘야겠네.

라나 네. 곱셈을 덧셈보다 먼저 하도록 정해진 규칙을 따르도록 해야 합니다. 이런 식으로 토큰들이 규칙에 따라 구조적으로 읽히는 과정을 구문 분석이라고 합니다.

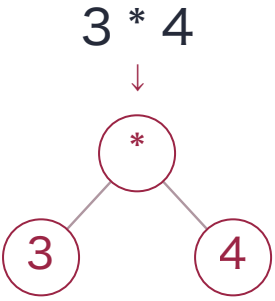


왼쪽과 오른쪽
대상 하나씩

라나 여기서 더하기와 곱하기에는 계산할 대상이 두 개씩 필요합니다. 기호의 왼쪽과 오른쪽에 하나씩입니다.

선생님 숫자가 들어갈 자리네.

라나 네. 식이 들어갈 수도 있습니다. 우선 곱셈부터 보면, 왼쪽에는 3, 오른쪽에는 4가 있습니다.

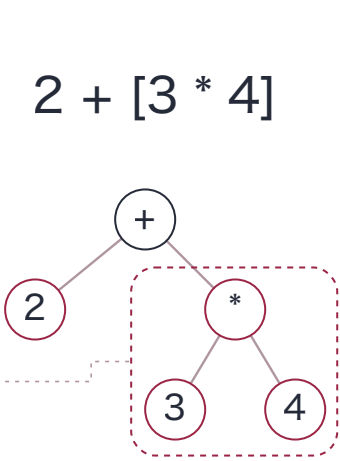


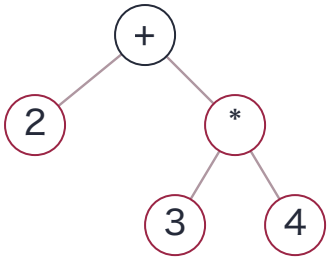
선생님 곱하기에 붙어 있는 두 숫자를 아래로 내려 그린 거구나.

라나 네. 이제 덧셈을 보겠습니다. 왼쪽에는 2가 있고, 오른쪽에는 방금 묶은 곱셈 전체가 들어갑니다.

선생님 오른쪽 자리에 숫자 하나 대신, 작은 그림이 통째로 들어가네.

라나 네. 이렇게 식의 구조를 나타낸 것을 구문 트리라고 합니다. 아직 계산한 것은 아니고, 무엇과 무엇을 계산할지 연결해 둔 겁니다.





구문 트리

선생님 더하기가 맨 위에 있는데, 곱하기를 먼저 해?

라나 네. 위의 덧셈을 하려면 오른쪽 값이 필요합니다. 그래서 그 아래의 3 곱하기 4부터 계산하는 겁니다.

3 * 4 → 12

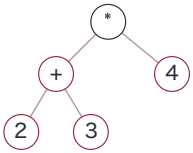
2 + 12 → 14

값으로 바꾸기

선생님 그 자리에 12가 들어가고, 마지막에 2를 더하면 14네.

라나 네. 식의 값을 구하는 것을 평가라고 합니다. 지금은 트리의 구조를 따라 직접 계산했습니다.

(2 + 3) * 4



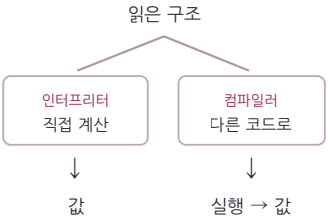
→ 20

선생님 괄호로 2 더하기 3을 묶으면?

라나 이번에는 덧셈 쪽을 먼저 묶습니다. 트리도 달라지고, 결과는 20이 됩니다.

라나 이처럼 읽어 낸 구조에 따라 계산을 수행하는 프로그램을 인터프리터라고 합니다.

라나 바로 계산하지 않고, 다른 형태의 코드로 옮길 수도 있습니다. 그 일을 하는 프로그램이 컴파일러입니다.



선생님 계산할 일을 다른 코드로 써 주는 거구나.

라나 네. 옮긴 코드를 실행하면 계산이 이루어집니다. 이번에는 식을 읽고 직접 계산하는 쪽을 만든 겁니다.

2 + 3 * 4 → 20

덧셈 우선 규칙

선생님 그런데 아까 곱셈부터 하기로 정한 것도 우리잖아. 덧셈부터 하는 계산기도 만들 수 있어?

라나 네. 덧셈을 먼저 묶도록 규칙을 바꾸면 됩니다.

선생님 그러면 2 더하기 3 곱하기 4가, 괄호 없이도 20이겠네?

라나 네. ……하지만 이건 선생님 전용으로 표시해 두겠습니다. 다른 분이 쓰시면 꽤 당황하시겠군요.

※ 여기서는 숫자·덧셈·곱셈·괄호만 있는 작은 언어를 다룹니다. 인터프리터가 반드시 트리를 직접 실행하는 것은 아니며, 컴파일과 해석을 함께 사용하는 구현도 있습니다.

스택으로 부족하다면

abc
aabbcc
aaabbbccc

수용

aabbc

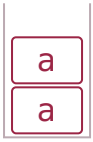
거부

라나 선생님. 이번에는 a, b, c가 이 순서로 나오면서, 개수도 모두 같은지 확인해 보겠습니다.

선생님 a와 b는 스택으로 맞춰 보면 되겠네. a를 쌓고, b가 나오면 하나씩 꺼내고.

라나 네. 그런데 a와 b를 다 맞추고 나면, 스택은 비어 있습니다.

a를 쌓고 b마다 꺼내는 방법



aa를 읽은 뒤

bb →

bb를 읽은 뒤

cc

비교할 개수는?

선생님 c도 두 개여야 하는데, 몇 개였는지 남아 있지 않네.

라나 맞습니다. 이 방법으로는 c와 비교할 개수를 남겨 두지 못합니다.

선생님 쌓는 방법을 다르게 하면 안 될까?

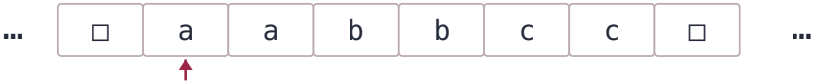
라나 이 방법만 실패한 것이라면, 다른 방법을 찾아볼 수 있겠지요. 하지만 이 문제는 스택 하나를 쓰는 푸시다운 오토마타로는 해결할 수 없다고 알려져 있습니다.

라나 개수에 제한을 두지 않고 모든 입력을 확인하려는 경우입니다. 증명은... 여기서 하기엔 페이지가 부족하네요

라나 이번에는 글자를 읽고 지나간 뒤에도, 다시 돌아와 표시할 수 있게 해보겠습니다.

□	a	a	b	b	c	c	□

※ 입력을 한 방향으로 읽고 스택 하나를 사용하는 표준 푸시다운 오토마타를 말합니다. 불가능성의 증명은 생략하며, 앞의 실패 예는 직관을 위한 것입니다.



상태: a를 찾는 중

선생님 이번에는 길게 늘어선 칸이네.

라나 테이프라고 부릅니다. 한 칸에 기호 하나를 적고, 헤드가 가리키는 칸을 읽습니다.

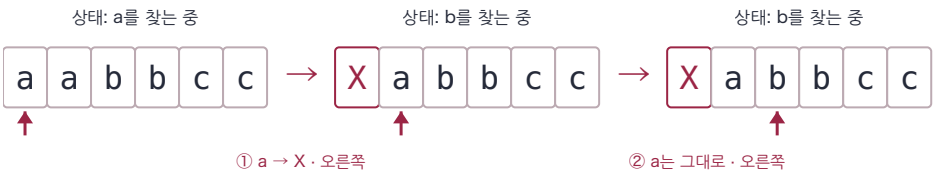
선생님 읽은 글자를 바꿔 써도 되는 거야?

라나 네. 그 칸에 기호를 쓰고, 왼쪽이나 오른쪽으로 한 칸 움직일 수 있습니다.

라나 동작 규칙도 정해 두어야 합니다. 지금의 상태와 읽은 기호에 따라, 무엇을 쓰고 어디로 움직일지, 다음 상태는 무엇인지 이 상태표에 적어 두었습니다.

상태표 · 첫 a와 b를 표시할 때 쓰는 규칙 일부

	현재 상태	읽은 기호	쓰는 기호	이동	다음 상태
①	a를 찾는 중	a	X	오른쪽	b를 찾는 중
②	b를 찾는 중	a	a	오른쪽	b를 찾는 중
③	b를 찾는 중	b	Y	오른쪽	c를 찾는 중



같은 a를 읽어도, 상태에 따라 동작이 달라집니다.

선생님 상태는 아까처럼 있고, 기억하는 곳이 테이프로 바뀐 거구나.

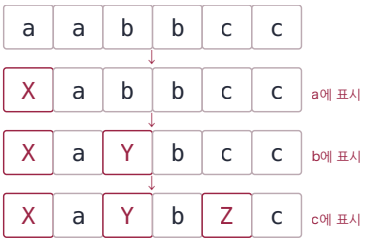
라나 **네. 테이프와 헤드, 그리고 상태에 따른 동작 규칙을 갖춘 계산 모형을 튜링 머신이라고 합니다.**

선생님 테이프 끝에 닿으면?

라나 이론상으로는 테이프 길이엔 제한이 없어서 필요한 만큼 빈 칸을 쓸 수 있습니다. 다만 멀리 있는 칸까지 가려면, 그만큼 움직여야 합니다.

라나 aabbcc로 해보겠습니다. 먼저 a, b, c가 순서대로 모여 있는지 확인합니다.

라나 그다음 아직 표시하지 않은 a, b, c를 하나씩 찾아, 각각 X, Y, Z로 바꿉니다.



문자를 찾아 표시하는 구간별 요약

선생님 한 개씩 짝지었다는 표시네.

선생님 왼쪽으로 돌아가는 것도 규칙에 적혀 있어?

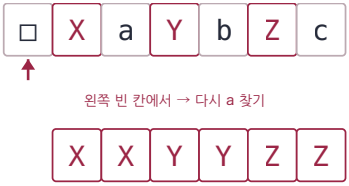
라나 네. c를 Z로 바꾸면 '왼쪽으로 돌아가는 중'으로 상태를 바꿉니다. 왼쪽의 빈 칸에 닿을 때까지 한 칸씩 돌아가는 겁니다.

라나 빈 칸에 닿으면 다시 a를 찾습니다. X는 이미 확인했으니 지나가고요.

왼쪽으로 돌아가 다시 시작하는 규칙 · 일부

현재 상태	읽기 → 쓰기	이동	다음 상태
c를 찾는 중	c → Z	왼쪽	왼쪽으로 돌아가는 중
왼쪽으로 돌아가는 중	s → s	왼쪽	왼쪽으로 돌아가는 중
왼쪽으로 돌아가는 중	□ → □	오른쪽	a를 찾는 중
a를 찾는 중	X → X	오른쪽	a를 찾는 중

s는 a, b, c, X, Y, Z 중 하나입니다. s → s는 읽은 기호를 그대로 둡니다.



선생님 두 번째까지 하고 나면, 전부 표시가 되네.

라나 네. a, b, c가 남지 않았으므로 받아들입니다. 도중에 짝이 부족하거나, 마지막에 b나 c가 남으면 거부합니다.

선생님 글자를 지워 없애지 않고, 확인한 자리에 표시를 남겼구나.

선생님 내가 손으로 따라 해도 같은 답이 나오겠네.

라나 네. 종이와 연필을 준비해 주시겠습니까? 제가 규칙을 읽어드리겠습니다.

선생님 설명 듣고 있었는데 어느샌가 내가 컴퓨터가 된 느낌이야.

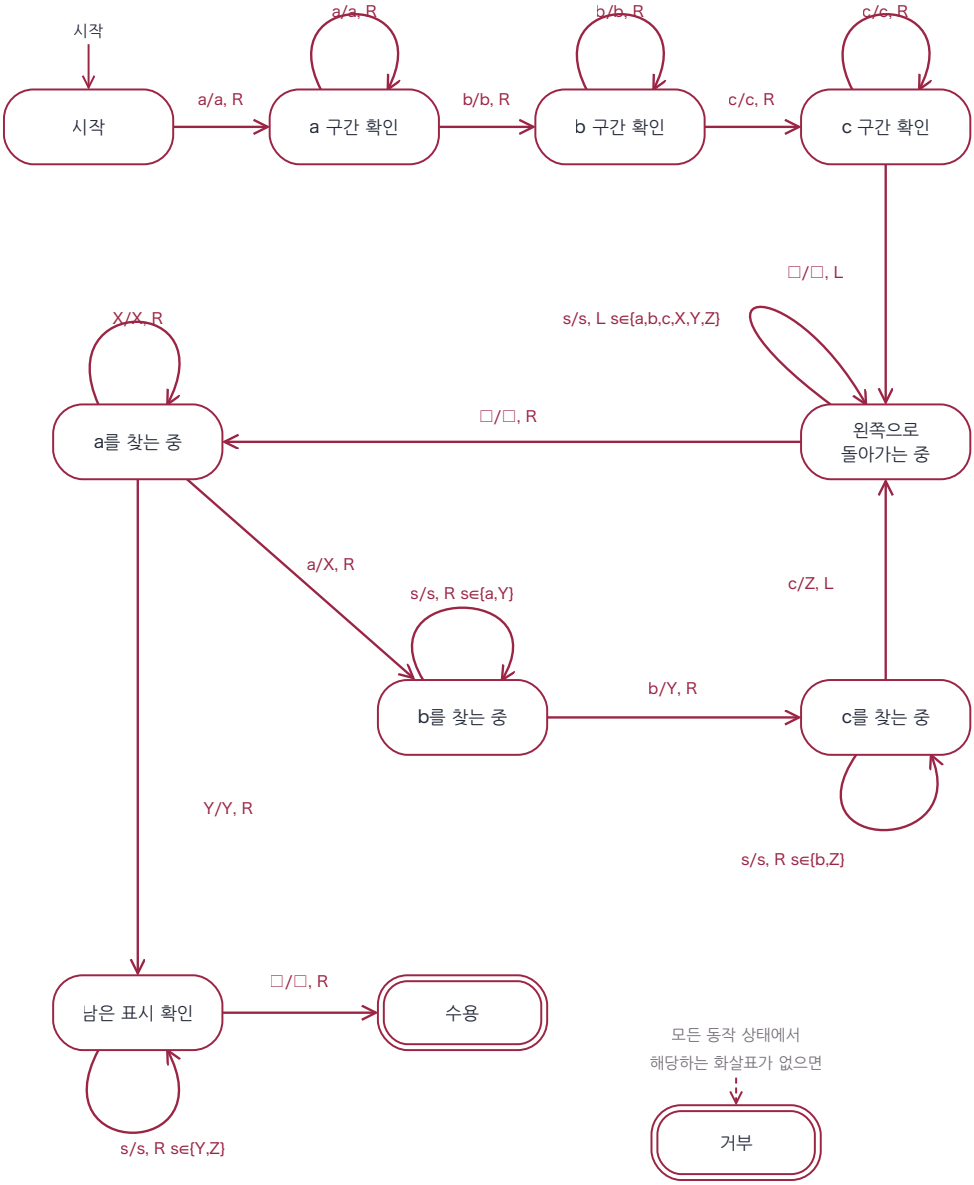
※ 테이프 변화는 구간별 요약입니다. 실제 머신은 한 번에 한 칸을 읽고 쓰며 이동합니다.

이 예시의 전체 상태 다이어그램

화살표: 읽은 기호 / 쓰는 기호, 이동 방향 · L = 왼쪽, R = 오른쪽

s/s는 기호를 그대로 둡니다. s∈{…}는 괄호 안의 각 기호에 대한 규칙을 묶은 표기입니다.

시작할 때 헤드는 입력의 첫 칸에 있습니다. 입력 양쪽의 나머지 칸은 모두 □입니다.

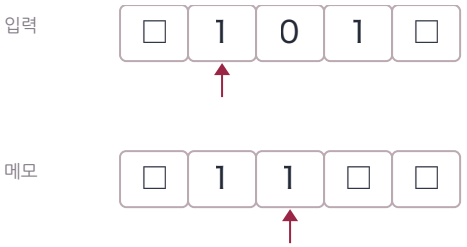


그려진 화살표가 없는 경우: 기호를 그대로 두고 오른쪽으로 한 칸 이동한 뒤 거부합니다.

수용·거부 상태에서는 멈춥니다. 빈 문자열도 시작 상태에서 거부합니다.

※ 입력은 a, b, c로만 이루어집니다. 여기서는 각 문자가 한 개 이상인 경우(n ≥ 1)를 다룹니다.

테이프를 더 늘리면?



라나 선생님. 이번에는 테이프를 하나 더 놓아보겠습니다. 하나는 입력을 읽고, 다른 하나는 메모하는 데 쓰면 어떨까요?

선생님 종이를 왔다 갔다 하는 일이 좀 줄겠네.

라나 네. 테이프마다 헤드를 두고 따로 움직일 수 있습니다. 이런 기계를 다중 테이프 튜링 머신이라고 합니다.

선생님 그러면 어떤 부분이 달라져?

라나 메모할 곳을 따로 쓸 수 있으니, 계산을 더 편하게 구성할 수 있습니다. 다만 테이프가 하나일 때는 못 하던 계산까지 할 수 있게 되는 건 아닙니다.

라나 두 테이프에 적힌 내용을, 이렇게 한 테이프에 나눠 적을 수도 있습니다. 각 헤드가 어디 있었는지도 표시해 두고요.

선생님 대신 표시된 자리들을 찾아다녀야겠네.

라나 네. 한쪽 기계가 한 번 움직이는 동안, 다른 쪽은 여러 번 움직여야 할 수 있습니다. 그래도 같은 계산을 따라 할 수는 있습니다.

선생님 할 수 있는 일이 늘어나는 건 아니구나.

라나 맞습니다. 반대로 테이프가 여러 개라면 하나만 써도 되겠지요. 서로의 계산을 흉내 낼 수 있다는 뜻에서, 이 두 모형은 튜링 동치라고 합니다.

※ 여기서 동치는 계산할 수 있는 범위에 관한 말입니다. 같은 계산을 하더라도, 필요한 계산 단계 수나 사용하는 기억 공간의 크기는 달라질 수 있습니다.

라나 이번에는 규칙을 바꿔보겠습니다. 같은 상태에서 같은 기호를 읽어도, 다음 동작이 여러 개라면 어떨까요?

선생님 어느 쪽으로 가야 하는데?

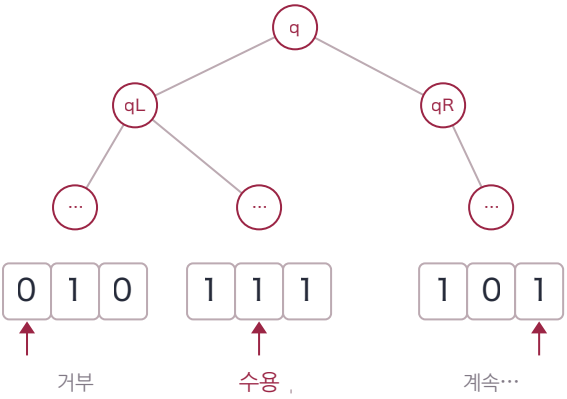
라나 가능한 선택마다 계산이 갈라진다고 생각해 주세요. 이렇게 선택이 갈라지는 모형을 비결정적 튜링 머신이라고 합니다.

현재 상태 q
읽은 기호 0

→ 0을 쓰고 왼쪽으로, 다음 상태 qL

→ 1을 쓰고 오른쪽으로, 다음 상태 qR

갈라진 뒤의 계산 · 개념도



선생님 한쪽에서는 맞다고 하고, 다른 쪽에서는 아니라고 하면?

라나 수용하는 가지가 하나라도 있으면, 그 입력을 받아들입니다. 모든 가지가 동의할 필요는 없습니다.

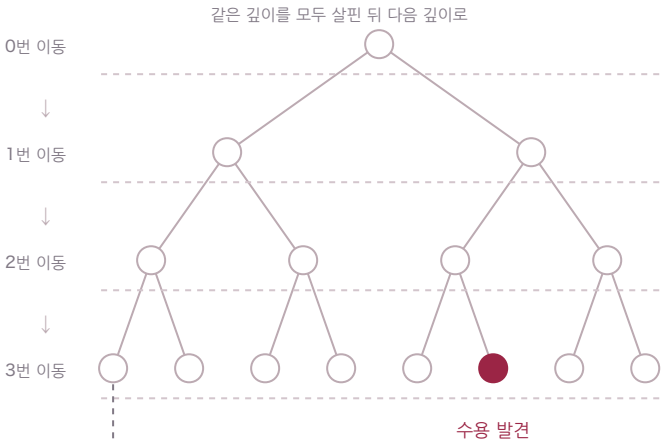
선생님 동전을 던져서 한쪽만 따라가면, 맞는 쪽을 놓칠 수도 있겠네.

라나 네. 여기서는 뽑힐 확률을 따지지 않습니다. 수용하는 계산이 하나라도 있는지를 보는 겁니다.

※ 수용하는 가지가 없다면 그 입력을 받아들이지 않습니다. 그렇다고 반드시 거부하고 멈추는 것은 아닙니다. 끝없이 이어지는 가지가 있을 수도 있습니다.

선생님 아까 기계로도 이걸 따라 할 수 있을까? 갈림길에서는 하나씩 골라야 하잖아.

라나 갈라진 계산들을 테이프에 기록해 두면 됩니다. 한 번 움직여서 갈 수 있는 곳을 모두 보고, 그다음에는 두 번, 세 번 움직여서 갈 수 있는 곳을 보는 식입니다.



선생님 한 길을 끝까지 가보고 돌아오는 건 아니네.

라나 네. 그 길이 끝없이 이어진다면 다른 길은 살펴보지도 못할 테니까요. 이렇게 차례로 살피면 수용하는 가지가 있을 때 언젠가는 찾을 수 있습니다.

선생님 하나씩 따라 해도, 받아들이는 입력은 같겠구나.

라나 네. 비결정적 튜링 머신도, 아까의 결정적 튜링 머신과 같은 언어를 인식할 수 있습니다. 다만 따라가야 할 계산이 아주 많아질 수 있습니다.

선생님 하나씩 하려니까 적어둘 게 점점 많아지네.

라나 네. 선생님이 한 분이셔도 따라 하실 수는 있습니다. ……종이를 더 가져오겠습니다.

선생님 쉬는 시간도 좀 넣어줘.

※ 비결정성은 실제 장치가 무한히 많은 계산을 동시에 끝낸다는 뜻이 아닙니다. 위 설명은 가능한 계산을 차례로 탐색하는 방식입니다.

얼마나 짧게 쓸 수 있을까요?

```
0000000000000000000000000
0000000000 … 0000000000
```

0이 10,000개

라나 선생님. 이 파일과 똑같은 파일을 만들려면, 0을 만 개 다 적어야 할까요?

선생님 복사하면 되지 않을까?

라나 원본을 가져다 쓰지 않고 만든다면요?

선생님 그러면…… 0을 만 번 쓰라고 시키면 되겠네.

라나 네. 결과를 전부 적는 것보다, 만드는 방법을 적는 쪽이 짧겠지요.

```
0을 10,000번 출력
그리고 종료
```

→

```
000…000
```

프로그램의 동작을 풀어 쓴 카드 — 같은 결과

선생님 같은 파일을 만드는 방법은 여러 가지겠네.

라나 네. 각각의 방법을 프로그램으로 적을 수 있습니다. 그중 가장 짧은 프로그램의 길이를 콜모고로프 복잡도라고 합니다.

선생님 얼마나 복잡하게 생겼는지가 아니라, 만드는 방법이 얼마나 짧은지구나.

라나 네. 여기서는 앞에서 본 튜링 머신을 생각해 주세요. 이번에는 테이프에 적힌 프로그램을 읽고 실행하도록 규칙을 정해 두겠습니다. 프로그램은 0과 1로 적고, 몇 글자인지 세겠습니다.

※ 프로그램의 길이는 모두 같은 표기 규칙으로 0과 1로 적어서 비교합니다. 특정 문자열만 짧게 만들려고 새 명령을 추가하거나, 실행하는 튜링 머신의 규칙을 바꾸지는 않습니다. 필요한 데이터도 글자 수에 포함하며, 프로그램은 별도 입력 없이 해당 문자열만 출력하고 멈춰야 합니다. 실행 시간은 세지 않습니다.

선생님 잘 찾아보면, 어떤 파일이든 짧게 만들 수 있지 않을까?

라나 그럴 수 있는지, 개수를 세어볼까요? 0과 1로 만들 수 있는 세 글자 문자열은 이렇게 여덟 개입니다.

만들 문자열 · 세 글자

000	001	010	011
100	101	110	111

더 짧게 적을 수 있는 것 · 일곱 개

빈 문자열	0	1	00
01	10	11	

선생님 그보다 짧게 적으려면 두 글자까지겠네.

라나 네. 아무것도 적지 않은 경우까지 세어도 일곱 개뿐입니다.

선생님 여덟 개를 하나씩 맡기기에는 하나가 모자라네.

라나 맞습니다. 한 프로그램이 출력하고 끝내는 결과는 하나입니다. 더 짧은 프로그램을 모든 문자열에 하나씩 줄 수는 없습니다.

선생님 일곱 개가 전부 쓸 만한 프로그램인 것도 아닐 테고.

라나 네. 그래서 만들 수 있는 결과는 더 적을 수도 있습니다. 네 글자, 다섯 글자로 늘려도 마찬가지입니다. 그보다 짧게 적을 수 있는 것들을 전부 모아도, 만들려는 문자열들에 하나씩 나눠 주기에는 늘 하나가 모자랍니다.

선생님 아무리 잘 찾아도, 더 짧게 못 만드는 문자열이 남겠구나.

라나 네. 그런 문자열을 비압축 문자열이라고 합니다.

※ 여기서 '비압축'은 정해 둔 기계에서 원래 문자열보다 짧은 프로그램으로 만들 수 없다는 뜻입니다. 겉보기에 불규칙하다는 것만으로 비압축 문자열이라고 판단할 수는 없습니다.

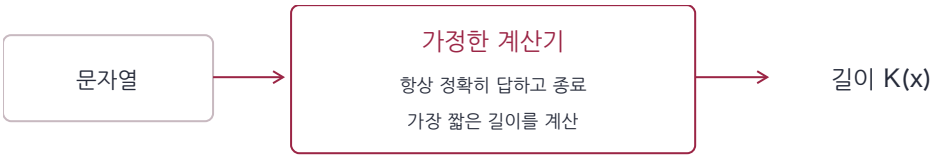
선생님 그럼 파일을 주면, 그걸 만드는 프로그램을 몇 글자까지 줄일 수 있는지도 알아낼 수 있어?

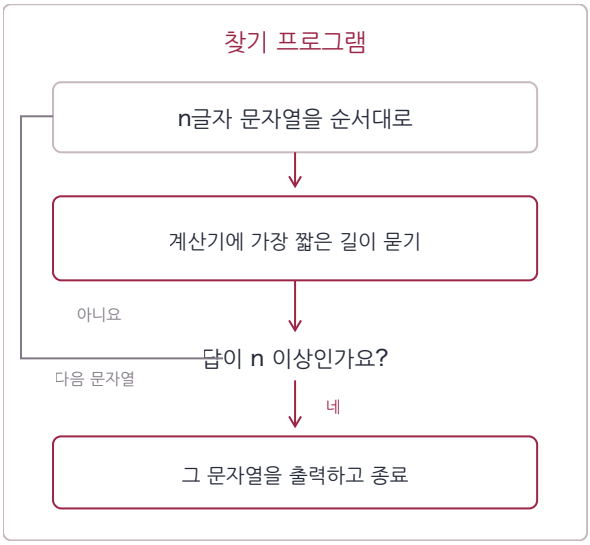
라나 어떤 파일이든 정확히 알려주는 프로그램은 만들 수 없습니다.

선생님 만드는 방법이 너무 많아서?

라나 그 때문만은 아닙니다. 계산기가 하나 있다고 생각해 볼까요?

라나 시간이 오래 걸려도 괜찮습니다. 어떤 문자열을 넣어도 가장 짧은 길이를 알려주고 끝내는 프로그램이 있다고 해보겠습니다.





라나 0과 1로 된 n글자 문자열들을 순서대로 넣어보겠습니다. 답이 n보다 작으면 넘어가고, n 이상이면 그 문자열을 출력하고 끝냅니다.

선생님 더 짧게 못 만드는 걸 찾는 거네. 그런데 꼭 찾을 수 있어?

라나 네. 앞에서 그런 문자열이 반드시 남는다고 했지요. 계산기도 매번 답하고 끝난다고 가정했으니, 언젠가는 찾습니다.

선생님 계산기가 있다면, 찾는 방법 자체는 어렵지 않네.

라나 네. 이제 이 찾기 프로그램의 길이를 세어보겠습니다.

※ K(x)는 앞에서 정한 기계에 대한 콜모고로프 복잡도입니다. 계산기는 실제로 만든 것이 아니라, 존재한다고 가정한 것입니다.

선생님 안에 넣은 계산기가 엄청나게 길 수도 있잖아.

라나 네. 그 프로그램의 글자 수도 전부 세어야 합니다. 그래도 한 번 만들어 두면, 찾을 문자열의 길이 n을 바꿀 때마다 계산기를 새로 만들 필요는 없습니다.

선생님 바꿔 적을 건 n뿐이네.

계산기 + 순서대로 찾는 방법 이 부분의 길이는 그대로	+	n을 적는 부분 8 → 1000

두 부분을 모두 센 것이 프로그램 전체 길이

라나 네. n도 0과 1로 적어 두겠습니다. 예를 들어 8은 1000, 16은 10000입니다. 값은 두 배인데, 적는 데는 한 글자만 더 필요하지요. 숫자의 끝을 구분하는 표시까지 넣어도, 충분히 큰 n에서는 전체 프로그램을 n글자보다 짧게 만들 수 있습니다.

찾아낸 문자열의 조건 가장 짧아도 n글자 이상 이보다 짧게 만들 수 없음	모순	방금 만든 찾기 프로그램 전체가 n글자 미만 같은 문자열을 출력하고 종료

n이 충분히 클 때, 두 조건은 함께 성립할 수 없습니다.

선생님 잠깐. 그 프로그램이 찾아낸 건, n글자보다 짧게 못 만든다는 문자열이잖아.

라나 네. 그런데 지금 그 문자열을 만드는 더 짧은 프로그램이 생겼습니다. 계산기가 항상 정확히 답한다는 가정이 맞을 수 없는 겁니다.

선생님 짧은 방법을 하나 찾는 것과, 그게 가장 짧다고 알아내는 건 다른 일이구나.

라나 네. 개별 문자열에서 최솟값을 알 수 있는 경우는 있지만, 모든 문자열의 최솟값을 계산하는 프로그램은 없습니다.

선생님 그럼 아까 이 카드에도 '가장 짧음'이라고 쓰면 안 되겠네.

라나 '짧은 방법'으로 적어 주세요. ……'가장' 한 단어를 붙이는 게 이렇게 어렵습니다.

※ 계산기와 찾는 절차를 적은 부분의 길이는 그대로입니다. 찾을 문자열의 길이를 두 배씩 늘려도, 그 길이를 0과 1로 기록하는 부분은 한 글자씩만 늘어납니다. 숫자를 구분하는 표시까지 포함해도, 충분히 긴 문자열을 찾게 하면 찾기 프로그램 전체가 그 문자열보다 짧아집니다.

왜 계산할 수 없다는 걸까요?

문자열 x를 출력하고 멈추는 가장 짧은 프로그램의 글자 수를 K(x)라고 쓰겠습니다. 프로그램은 앞에서 정한 기계에서 실행하며, 같은 규칙으로 0과 1로 적어 비교합니다.

① 가장 짧은 길이를 알려주는 계산기를 가정합니다.

어떤 문자열 x를 넣어도 정확한 K(x)를 답하고 끝내는 계산기가 있다고 해봅시다. 빠르게 답할 필요는 없습니다. 이 계산기로 길이가 n인 문자열을 순서대로 살펴, K(x)가 n 이상인 첫 문자열을 출력하고 끝냅니다.

② 찾을 문자열은 반드시 있습니다.

0과 1로 된 문자열의 개수를 비교하면 다음과 같습니다. 더 짧은 프로그램 하나는 결과를 하나만 만들 수 있으므로, 모든 n글자 문자열을 더 짧은 프로그램으로 만들 수는 없습니다.

길이가 n인 문자열

$$2^n$$

$>$

그보다 짧은 문자열

$$1 + 2 + \cdots + 2^{n-1} = 2^n - 1$$

만들 결과가 더 많으므로, K(x) ≥ n인 문자열이 남습니다.

③ 찾는 방법도 하나의 프로그램입니다.

계산기와 탐색 절차, 숫자를 읽는 규칙까지 포함한 고정 부분의 길이를 c라고 하겠습니다. 여기에 n을 적어 넣으면, 별도 입력 없이 x를 찾아 출력하는 프로그램 P이 됩니다. n 자체를 적는 데 n글자가 필요한 것은 아닙니다.

n의 이진 표기가 d글자라면, 각 글자를 두 번 쓰고 끝에 01을 붙여 숫자의 끝을 구분할 수 있습니다. 이 방식으로 n을 기록하는 데 2d + 2글자가 듭니다.

찾기 프로그램의 길이: $|P| \le c + 2d + 2$

찾을 문자열 길이 n	n을 0과 1로 쓴 길이 d	n의 기록 길이 2d + 2
16	5	12
256	9	20
65,536	17	36

표는 n을 기록하는 부분만 셉니다. 계산기와 탐색 코드의 길이 c도 반드시 더해야 합니다.

④ 충분히 큰 n에서 두 조건이 충돌합니다.

n은 두 배씩 커져도, n을 이진수로 적는 자릿수 d는 하나씩만 늘어납니다. 따라서 c가 아무리 큰 고정값이어도, 충분히 큰 n에서는 c + 2d + 2가 n보다 작아집니다. 그런데 P도 x를 만드는 프로그램이므로, 가장 짧은 길이 K(x)는 P의 길이보다 클 수 없습니다.

찾을 때 요구한 조건

$$K(x) \ge n$$

그 문자열을 만드는 프로그램

$$K(x) \le |P| < n$$

모순 때문에 버려야 하는 것은 "모든 문자열의 K(x)를 정확히 답하고 끝내는 계산기가 있다"는 가정입니다. 특정 문자열의 짧은 프로그램을 찾거나, 개별 사례에서 최솟값을 알아낼 수 없다는 뜻은 아닙니다.

끝나기는 하는데요.

선생님 아까는 시간이 오래 걸려도 괜찮다고 했지?

라나 네. 그래도 만들 수 없는 계산기가 있다는 이야기였습니다. 이번에는 끝낼 수 있는 계산이 얼마나 오래 걸리는지 보겠습니다.

선생님 이번엔 기다리면 되긴 하는 거지?

라나 네. 다만 선생님이 기다리실 수 있는 시간 안에 끝날지는 따로 살펴봐야 합니다.

한 글자씩, 끝까지 한 번

0 1 0 1 1 0 — 6글자

0 1 0 1 1 0 0 1 0 1 1 0 — 12글자

기억할 상태

짝수 / 홀수

1을 읽으면 전환

라나 처음에 1의 개수가 짝수인지 확인했던 방법을 기억하시나요? 글자를 하나 읽을 때마다 상태를 바꿀지 정했습니다.

선생님 짝수인지 홀수인지만 기억하면 됐지.

라나 네. 문자열이 두 배로 길어지면, 읽어야 할 글자도 두 배가 됩니다.

선생님 컴퓨터가 얼마나 빠른지 몰라도, 할 일이 얼마나 늘어나는지는 알 수 있구나.

라나 맞습니다. 입력이 길어질 때 필요한 계산 단계 수가 늘어나는 것을 시간 복잡도라고도 부릅니다.

※ 여기서 입력의 길이는 문제를 정해진 방식으로 적었을 때의 글자 수입니다. 같은 길이의 입력 중 가장 많은 단계가 필요한 경우를 기준으로 보겠습니다. 기계의 동작 한 번을 한 단계로 셉니다.

선생님 어떤 계산이든 길이가 두 배면, 시간도 두 배쯤 걸려?

라나 그렇지는 않습니다. 입력의 글자들을 서로 하나씩 짝지어 살피면, 비교할 쌍은 길이의 제곱만큼 생깁니다.

선생님 두 배로 길어졌는데 네 배나 살펴봐야 해?

라나 네. 반면 매번 두 갈래로 나뉜 선택을 전부 살핀다면, 선택을 한 번 더 할 때마다 경우의 수가 두 배가 됩니다.

증가 형태를 비교한 예

입력 길이 n	n	n^2	2^n
10	10	100	1,024
20	20	400	1,048,576
30	30	900	1,073,741,824

선생님 길이를 조금 늘렸을 뿐인데, 저쪽은 벌써 숫자가 엄청 커졌구나.

라나 네. 길이가 두 배일 때 네 배, 세제곱이라면 여덟 배가 되는 식이지요. 이렇게 입력 길이의 몇 제곱에 일정한 수를 곱한 횟수 안에서 끝낼 수 있으면, 다항 시간이라고 합니다.

선생님 마지막 것은 한 글자 늘 때마다 경우의 수가 두 배가 됐고.

라나 네. 그렇게 늘어나는 것을 지수적인 증가라고 합니다.

선생님 그러면 다항 시간에 끝나는 방법이 있으면 되는 거야?

라나 네. 다항 시간에 '예'인지 '아니요'인지 판정할 수 있는 문제들을 P 문제라고 부릅니다.

※ P는 프로그램이 아니라 문제들의 모음입니다. 다항 시간은 모든 입력에서 성립하는 단계 수의 상한에 관한 말입니다. 다항 시간이라고 언제나 실용적으로 빠른 것은 아니며, 표의 세 형태가 가능한 모든 증가 형태를 뜻하지는 않습니다.

각 카드 한 번씩 · 목표 합 16

3	5	8	11

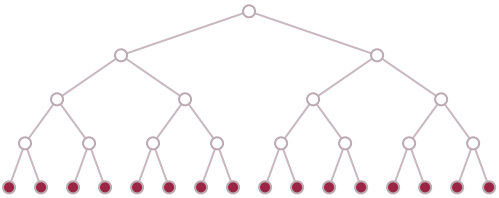
라나 선생님. 여기서 카드를 몇 장 골라 합을 16으로 만들 수 있을까요? 각 카드는 한 번씩만 쓸 수 있습니다.

선생님 3하고 8이면 11이고…… 아, 5하고 11을 고르면 되겠다.

라나 네. 고르신 두 장을 더해 보면 바로 확인할 수 있습니다.

선생님 고를 때는 이것저것 해봤는데, 맞는지 보는 건 덧셈 한 번이면 끝이야?

카드마다 선택 / 비선택



네 장 → 16가지 선택

고른 카드 확인

5 + 11 = 16

'있다'는 답의 근거

라나 네. 카드가 많아지면 어떨까요? 모든 조합을 살피는 방법에서는, 카드가 한 장 늘 때마다 경우의 수가 두 배가 됩니다.

선생님 누가 고른 카드를 보여주면, 그 카드들만 더해 보면 되는데.

라나 네. 주어진 카드인지, 한 장을 여러 번 쓰지는 않았는지도 확인하면 됩니다. 숫자가 길어지면 덧셈에도 시간이 더 들지만, 이런 확인은 입력 길이에 대한 다항 시간 안에 할 수 있습니다.

선생님 찾는 것하고, 찾아온 걸 확인하는 건 일이 꽤 다르구나.

라나 맞습니다. 여기서 묻는 문제는 '합을 맞추는 선택이 있는가?'입니다. 선생님이 고른 카드들이 '있다'는 답의 근거가 됩니다.

※ 카드의 개수와 숫자를 적는 길이는 다릅니다. 입력 길이에는 모든 카드의 숫자와 목표 합을 적는 데 필요한 글자 수도 포함합니다. 이 작은 예시만으로 모든 경우가 어렵다고 결론 내리지는 않습니다.

선생님 반대로, 아무리 골라도 안 되는 경우는?

라나 그때는 맞는 카드 조합을 보여줄 수 없습니다. 지금 이야기한 확인 방법은 '있다'는 답을 뒷받침하는 방법입니다.

'있다'는 답 + 선택한 카드 목록

주어진 카드인가? → 중복 사용은 없는가? → 합이 목표와 같은가?

선생님 '못 찾았어요'라는 말만으로는, 정말 없다고 할 수는 없겠지.

라나 네. 이렇게 '예'라는 답에 대해, 다항 시간 안에 확인할 수 있는 근거가 있는 문제들을 NP 문제라고 합니다. 근거의 길이도 입력 길이에 대한 다항식으로 제한합니다.

선생님 확인할 자료부터 끝도 없이 길면 곤란하겠지.

$P \subseteq NP$

P 문제는 모두 NP 문제

같은지는 아직 모릅니다.

선생님 아까 P 문제는 답을 직접 구할 수 있는 문제들이었지? 그럼 그것들도 NP 문제야?

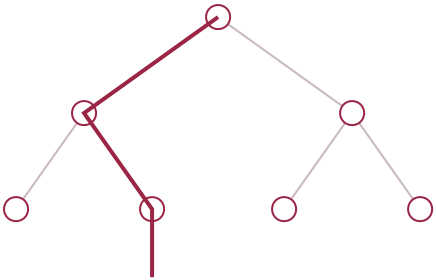
라나 네. 직접 판정해서 확인하면 되니까요. P 문제는 모두 NP 문제이기도 합니다.

선생님 그럼 NP 문제라고 해서 전부 오래 걸리는 문제는 아니구나.

라나 맞습니다. NP는 '다항 시간에 풀 수 없다'는 뜻이 아닙니다. 이름의 N은 앞에서 배운 비결정성에서 왔습니다.

※ NP의 근거는 '예'인 입력에는 적어도 하나 있고, '아니요'인 입력에는 확인을 통과하는 근거가 없어야 합니다. '아니요'라는 답에도 같은 방식의 짧은 근거가 있다고 요구하는 정의는 아닙니다.

비결정적 모형 · 선택과 확인



선택 구조의 일부를 표시한 개념도

선생님 카드마다 고를지 말지를 갈라놓으면, 아까의 계산 나무처럼 그릴 수 있겠다.

라나 네. 각 가지에서 선택을 마친 뒤 확인하면 됩니다. 맞는 선택이 있는 가지 하나라도 수용하면, '예'라는 답입니다.

라나 NP 문제는 이렇게 비결정적 튜링 머신으로 다항 시간에 판정할 수 있습니다. 어느 가지를 따라가도, 선택하고 확인하는 과정이 다항 시간 안에 끝나도록 하는 겁니다.

선생님 그럼 그런 기계로 컴퓨터를 만들면 되지 않을까? 하나씩 찾는 것보다 빠를 것 같은데.

라나 키보토스에서 저희가 쓰는 컴퓨터는 그렇게 움직이지 않습니다. 비결정적 튜링 머신은 가능한 선택을 가지로 나누어 생각하는 이론상의 모형입니다.

비결정적 모형의 시간

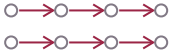
가장 긴 경로의 단계 수

모든 경로의 합이 아닙니다.

컴퓨터가 차례로 따라 한다면

①

②

③ 

여러 경로를 확인하는 시간이 듭니다.

선생님 종이에 갈림길을 그렸다고, 컴퓨터가 저절로 모든 길을 가주는 건 아니구나.

라나 네. 앞에서는 그 가지들을 하나씩 따라가는 방법을 보았지요. 가지 하나는 짧아도, 전부 따라가려면 오래 걸릴 수 있습니다.

선생님 그럼 우리가 쓰는 컴퓨터에서는, 꼭 그렇게 하나씩 해봐야 해?

라나 모든 가지를 살피지 않고 답을 구하는 다른 방법이 있을 수도 있습니다. NP 문제를 모두 결정적 튜링 머신으로도 다항 시간에 판정할 수 있는지 묻는 것이 P 대 NP 문제입니다.

선생님 그건 아직 모르는 거야?

라나 네. 아직 어느 쪽인지 밝혀지지 않았습니다.

P = NP ?

확인이 빠르면, 판정도 빠를까요?

'빠르다'의 기준은 다항 시간

선생님 계산할 수 없는 문제와는 다르구나. 할 수는 있는데, 얼마나 빨리 할 수 있는지는 아직 모르는 거지?

라나 네. 여기서는 후보가 유한해서, 전부 확인하면 언젠가는 끝납니다. 그보다 효율적인 방법이 있는지는 별개의 질문입니다.

선생님 이제 끝난 줄 알았는데, 마지막 문제가 아직 안 풀린 문제라니.

라나 선생님께 숙제로 드리려던 건 아닙니다.

선생님 다행이다. 그럼 오늘은 여기까지?

라나 네. 다음에는 답을 찾는 쪽도 같이 해보시겠어요?

※ 여기서는 앞의 결정적 튜링 머신으로 다항 시간에 판정하는 P와, 모든 가지가 다항 시간 안에 멈추는 비결정적 튜링 머신으로 판정하는 NP를 비교했습니다. 5챕터의 '계산할 수 있는 범위가 같다'는 사실만으로 '같은 시간 안에 계산할 수 있다'고 결론 낼 수는 없습니다.

라나
……생각보다 오래 이야기했네요.
선생님
벌써 시간이 이렇게 됐구나.
당번도 끝날 시간이야.

라나
아직 못 보여드린 것도 있는데…….
선생님
오늘 고마웠어. 덕분에 많이
배웠어.

선생님
그럼 다음에 이어서 들을까?
라나
네. 다음 당번 때, 이어서
알려드릴게요.

참고자료 · 출처

학습 내용

MIT OpenCourseWare
6.045J Automata, Computability, and Complexity (2011)
https://ocw.mit.edu/courses/6-045j-automata-computability-and-complexity-spring-2011/

OpenDSA
Pushdown Automata
https://opendsa-server.cs.vt.edu/ODSA/Books/PIFLA/html/PDA.html

Cornell University · Andrew Myers
CS 4120 Lecture Notes
https://www.cs.cornell.edu/courses/cs4120/2026sp/notes/

Robert Nystrom
Crafting Interpreters: Scanning / Parsing Expressions / Evaluating Expressions
https://craftinginterpreters.com/contents.html

York College of Pennsylvania
CS 340, Lecture 7: Turing Machines (2016)
https://ycpcs.github.io/cs340-fall2016/lectures/lecture07.html

Duke University
COMPSCI 334: Section — Turing Machines
https://courses.cs.duke.edu/cps140/compsci334/spring23/lects/sectntmS.pdf

Pramod Ganapathi · Stony Brook University
Theory of Computation (Turing Machines)
https://www3.cs.stonybrook.edu/~pramod.ganapathi/doc/theory-of-computation/TuringMachines.pdf

MIT 6.045
Lecture 9: Turing Machines (2020)
https://people.csail.mit.edu/rrw/6.045-2020/lec9-before-class.pdf

Nicolas Resch · Carnegie Mellon University
CS 252, Lecture 4: Kolmogorov Complexity (2020)
https://www.cs.cmu.edu/~venkatg/teaching/15252-sp21/notes/Kolmogorov-Complexity.pdf

Clay Mathematics Institute
P vs NP
https://www.claymath.org/millennium/p-vs-np/

원작 · 캐릭터

ブルーアーカイブ -Blue Archive-
캐릭터·세계관, 공식 스탠딩 일러스트 및 게임 내 선생님 그림
https://bluearchive.jp/

Blue Archive Utils · 캐릭터 소개·보이스 대사
https://blue-utils.me/student/rana?lang=ja https://blue-utils.me/tool/voice

ブルーアーカイブ攻略有志Wiki · ラナ
https://bluearchive.wikiru.jp/?ラナ